ĐỀ BÀI BÀI TẬP THỰC HÀNH — Spring Boot Practice Project

Gửi sinh viên: Làm việc với project `practice-project` (quản lý User, đăng ký/đăng nhập JWT, REST API). Hoàn thành các phần dưới đây và nộp theo hướng dẫn của giảng viên.

CÁC CHỦ ĐỀ KIẾN THỨC (Tổng quan)

Đề bài bao gồm toàn bộ các chủ đề sau để sinh viên nắm kiến thức Spring Boot cần thiết:

Chủ đề	Nội dung trong đề bài
AuthN & AuthZ	Đăng ký, đăng nhập JWT, role USER/ADMIN, @PreAuthorize, 401/403
REST & User CRUD	CRUD User, phân trang/lọc/sắp xếp, DTO, HTTP status chuẩn
Entity, JPA & Repository	Entity User, JpaRepository, truy vấn theo điều kiện
Exception & Validation	@RestControllerAdvice, ApiError thống nhất, Bean Validation (@Valid, @NotBlank, @Email...)
Property & Config	application.yml, profiles (dev/prod), @ConfigurationProperties (JWT)
Logging	Request ID (MDC), pattern log có requestId, mức INFO/WARN/ERROR
Testing	Unit test (Mockito), Repository (@DataJpaTest), Controller (MockMvc, 401/403)
CORS & Swagger	CORS cho frontend, Swagger UI, Bearer JWT trong OpenAPI
DB Migration & Seed	Liquibase hoặc Flyway: schema versioned, seed data (admin, roles)

Yêu cầu môi trường

- JDK 17+
- Maven 3.8+
- IDE: IntelliJ IDEA / VS Code / Eclipse

Cấu trúc project (gợi ý)

```
src/main/java/com/ou/practice/
├── config/           # SecurityConfig, JwtProperties, OpenApiConfig,
DataLoader
├── controller/       # AuthController, UserController, AdminController
├── dto/              # RegisterRequest, LoginRequest, AuthResponse,
UserResponse, ApiError, ...
├── entity/           # User, Role
├── exception/        # ResourceNotFoundException,
```

```
DuplicateResourceException, GlobalExceptionHandler
├── filter/           # RequestIdFilter (MDC requestId)
├── repository/       # UserRepository (JPA)
├── security/         # JwtService, JwtAuthenticationFilter,
UserDetailsServiceImpl, EntryPoint, AccessDeniedHandler
├── service/          # AuthService, UserService, UserMapper
└── PracticeApplication.java
```

PHẦN 1 — AUTHENTICATION & AUTHORIZATION (Đăng nhập, phân quyền)

1.1 Đăng ký và đăng nhập

- **Đăng ký:** Endpoint `POST /api/auth/register` nhận username, email, password. Tạo user mới, gán role mặc định `USER`. Mật khẩu phải được mã hóa (BCrypt), **không** lưu plain text. Response 201 và thông tin user (id, username, email, role), **không** trả password.
- **Đăng nhập:** Endpoint `POST /api/auth/login` nhận username (hoặc email) + password. Nếu đúng, trả về access token (JWT) và (tùy chọn) refresh token. Nếu sai, trả 401 và message phù hợp.
- **Thông tin user hiện tại:** Endpoint `GET /api/auth/me` (cần JWT) trả thông tin user đang đăng nhập (id, username, email, role).
- Các URL `/api/auth/**` là công khai (không cần token), **trừ** `/api/auth/me` nếu bạn chọn bảo vệ. Các API khác (trừ public) yêu cầu header `Authorization: Bearer <token>`.

1.2 Phân quyền theo role

- User đăng ký mới có role `USER`. Cần có ít nhất một user role `ADMIN` (tạo khi khởi động: DataLoader, script, hoặc **seed trong Liquibase/Flyway**).
- `/api/admin/**` chỉ cho role `ADMIN`. Ví dụ: `GET /api/admin/users` — danh sách user không phân trang (dành cho ADMIN).
- `/api/users/**` cho cả `USER` và `ADMIN`.
- Một số thao tác chỉ ADMIN: ví dụ xóa user (`DELETE /api/users/{id}`), tạo user qua `POST /api/users`. Dùng `@PreAuthorize("hasRole('ADMIN')")` trên method tương ứng.
- **401 Unauthorized:** khi không gửi token hoặc token sai/hết hạn.
- **403 Forbidden:** khi đã đăng nhập nhưng không đủ quyền (ví dụ USER gọi API chỉ dành cho ADMIN).

1.3 (Mở rộng) Refresh token

- Endpoint `POST /api/auth/refresh` nhận refresh token, trả về access token mới.

1.4 (Mở rộng) Cập nhật role

- Endpoint `PATCH /api/users/{id}/role` chỉ cho ADMIN, cho phép đổi role của user.

Câu hỏi ngắn (có thể trả lời trong báo cáo):
Khi gọi `GET /api/users` mà không gửi header `Authorization`, response mong đợi là gì (mã HTTP và ý nghĩa)? Phân biệt 401 và 403 trong REST API.

PHẦN 2 — REST API CRUD USER

2.0 Entity, JPA & Repository (nền tảng dữ liệu)

- **Kiến trúc lớp:** Controller → Service → Repository; dùng `@RestController`, `@Service`, `@Repository`. Service gọi Repository, không lộ entity ra ngoài API (dùng DTO).
- **Entity User:** Các trường id, username, email, passwordHash, role, createdAt (và updatedAt nếu có). Dùng annotation JPA (@Entity, @Table, @Column). Không lưu mật khẩu plain text. Role có thể là enum hoặc bảng riêng.
- **UserRepository:** Kế thừa `JpaRepository<User, Long>`. Có các method cần thiết: findByUsername, findByEmail, existsByUsername, existsByEmail; và method hỗ trợ phân trang + lọc (ví dụ findByUsernameContainingIgnoreCaseOrEmailContainingIgnoreCaseAndRole với Pageable, hoặc dùng Specification/QuerydsI).
- Đảm bảo cấu hình datasource (H2 in-memory cho dev) trong application.yml; có thể dùng profile để tách dev/prod.

2.1 Danh sách và chi tiết

- **GET /api/users:** Danh sách user có **phân trang**, **lọc**, **sắp xếp**.
  - Tham số: `page` (mặc định 0), `size` (mặc định 10, tối đa 100), `sort`, `order` (asc/desc), `search` (tìm theo username hoặc email), `role` (USER/ADMIN).
  - Response theo chuẩn phân trang: `content`, `page`, `size`, `totalElements`, `totalPages`, `first`, `last`.
- **GET /api/users/{id}:** Chi tiết một user. Có dữ liệu → 200 và body (UserResponse). Không tồn tại → 404 và body lỗi thống nhất.

2.2 Tạo và cập nhật

- **POST /api/users:** Chỉ ADMIN. Tạo user mới (có thể gán role). Validate request (username, email, password). Trùng username/email → 409 (hoặc 400) với message rõ. Thành công → 201, header `Location: /api/users/{id}`, body UserResponse. Mật khẩu phải hash trước khi lưu.
- **PUT /api/users/{id}:** Cập nhật toàn bộ. Chỉ chủ sở hữu (user đang đăng nhập) hoặc ADMIN. Không tồn tại id → 404. Validation lỗi → 400. Thành công → 200 và UserResponse.
- **PATCH /api/users/{id}:** Cập nhật một phần (chỉ gửi field cần đổi). Chỉ chủ sở hữu hoặc ADMIN. Quy tắc 404/400/200 tương tự.

2.3 Xóa

- **DELETE /api/users/{id}:** Chỉ ADMIN. Thành công → 204 (hoặc 200).

2.4 DTO và validation

- Dùng DTO cho request/response, không lộ entity ra ngoài.
- Request: validation với Bean Validation (`@NotBlank`, `@Email`, `@Size`, …). Ví dụ: username 3–50 ký tự, email hợp lệ, password tối thiểu 8 ký tự (và pattern nếu có yêu cầu).
- Response (UserResponse): id, username, email, role, createdAt; **không** trả password hay passwordHash.

2.5 Format lỗi thống nhất

- Dùng `@RestControllerAdvice` và `@ExceptionHandler` để trả lỗi thống nhất.

- Format gợi ý: `error`, `message`, `details` (danh sách lỗi theo field nếu là validation), `path`, `timestamp`.
- Validation lỗi → 400. Không tìm thấy resource → 404. Trùng username/email → 409 (hoặc 400).
- (Exception & Validation: xử lý MethodArgumentNotValidException, custom exception như ResourceNotFoundException, DuplicateResourceException.)

Câu hỏi ngắn: Tại sao nên dùng DTO thay vì trả trực tiếp Entity trong REST API?

PHẦN 3 — LOGGING (Request ID và log có cấu trúc)

3.1 Request ID (MDC)

- Mỗi HTTP request có một **request ID** duy nhất (UUID hoặc tương đương).
- Request ID được đưa vào **MDC** và xuất hiện trong mọi dòng log xử lý request đó (cấu hình pattern log có `[%X{requestId}]`).
- Trả request ID trong response header `X-Request-ID` để client có thể cung cấp khi báo lỗi.
- Sau khi xử lý request xong, clear MDC (trong filter/interceptor) để tránh rò rỉ giữa các request.

3.2 Mức log và nội dung

- **INFO:** Request vào/ra endpoint (method, path, status, thời gian xử lý ms). Sự kiện nghiệp vụ: đăng nhập thành công, tạo/cập nhật/xóa user (không ghi password/token).
- **WARN:** Đăng nhập thất bại (sai mật khẩu), không ghi password.
- **ERROR:** Exception trong `@RestControllerAdvice`, có requestId và (tùy môi trường) stack trace.
- **Không** ghi password, token (JWT) đầy đủ vào log; có thể mask hoặc bỏ qua.

Câu hỏi ngắn: Tại sao cần clear MDC sau khi xử lý request xong?

PHẦN 4 — TESTING

4.1 Unit test Service

- **AuthService:** Mock UserRepository, PasswordEncoder, JwtService. Test: (1) đăng ký thành công, password được encode, không trả password; (2) đăng ký trùng username/email → exception hoặc lỗi, không gọi save; (3) đăng nhập đúng → trả token; (4) đăng nhập sai mật khẩu → exception.
- **UserService:** Mock UserRepository. Test: findById có/không có user; create request hợp lệ → save và trả response có id; update/delete với id tồn tại và không tồn tại. Đảm bảo response không chứa password/passwordHash.

4.2 Test Repository (JPA)

- Dùng `@DataJpaTest`, H2 in-memory. Test: save user, findById, findByUsername/findByEmail (hoặc existsByEmail). Có thể dùng `@Sql` để nạp data mẫu.

4.3 Test Controller (MockMvc)

- Test ít nhất: một endpoint trả 200/201 (ví dụ GET /api/users/1 có data, POST /api/auth/register hợp lệ); một endpoint trả 400 hoặc 404 (validation, không tìm thấy).

- **Có JWT:** Một test gọi API bảo vệ **không** có header Authorization → 401. Một test gọi endpoint chỉ ADMIN với token USER → 403; với token ADMIN → 204/200 (ví dụ DELETE /api/users/{id}).

Có thể dùng token test (cùng secret, thời hạn dài) hoặc mock filter set SecurityContext với user có role USER/ADMIN.

PHẦN 5 — CẤU HÌNH ỨNG DỤNG (Property & Config)

5.1 application.yml

- Cấu hình server port (nếu cần), datasource (url, driver, username, password) cho H2.
- Cấu hình JPA: hibernate ddl-auto — khi **dùng Liquibase/Flyway** thì đặt `validate` (hoặc `none`) để schema do migration quản lý; khi chưa dùng migration có thể dùng `update`/`create-drop` cho dev.
- Cấu hình logging: level root và theo package (ví dụ `logging.level.com.ou.c3=DEBUG` cho dev).

5.2 Profiles (dev / prod)

- Profile `dev`: H2 in-memory, ddl-auto=update hoặc create-drop, log level DEBUG, có thể hardcode JWT secret cho tiện.
- Profile `prod`: Datasource MySQL (hoặc từ biến môi trường), `spring.jpa.hibernate.ddl-auto=validate`, JWT secret lấy từ biến môi trường. Có thể giảm log level (INFO). **Schema và seed do Liquibase/Flyway quản lý.**
- Chạy với profile: `mvn spring-boot:run -Dspring-boot.run.profiles=dev` hoặc `--spring.profiles.active=prod`.

5.3 @ConfigurationProperties cho JWT

- Tạo class (ví dụ `JwtProperties`) với `@ConfigurationProperties(prefix = "app.jwt")` để đọc từ application.yml: secret, expiration (thời hạn access token), refreshExpiration (nếu có).
- Trong application.yml: `app.jwt.secret=...`, `app.jwt.expiration=...`. Ở prod dùng `${JWT_SECRET}` từ môi trường.
- Inject `JwtProperties` vào `JwtService` thay vì dùng `@Value` rời rạc.

Câu hỏi ngắn: Lợi ích của @ConfigurationProperties so với @Value cho nhiều thuộc tính liên quan (như JWT)?

5.4 DB Migration & Seed (Liquibase hoặc Flyway)

Có nên dùng Liquibase? Có. Dùng công cụ migration giúp schema và dữ liệu khởi tạo được **versioned**, có thể tái tạo giống nhau trên mọi môi trường (dev/test/prod). Spring Boot hỗ trợ sẵn **Liquibase** và **Flyway**; chọn một trong hai.

	Liquibase	**Flyway**
Định dạng	XML, YAML, JSON hoặc SQL	SQL (hoặc Java callback)
Rollback	Có (khai báo rollback)	Không có sẵn (phải viết migration ngược)
Preconditions	Có (chạy changeSet khi thỏa điều kiện)	Có (trong SQL hoặc callback)

	Liquibase	Flyway
Phù hợp	Cần đa dạng format, làm việc với team quen Liquibase	Ưu tiên "chỉ SQL", đơn giản, dễ đọc

Gợi ý: Với project thực hành, **Liquibase** hoặc **Flyway** đều được. Nếu muốn một công cụ duy nhất, **Liquibase** cho phép XML/YAML/SQL và rollback; **Flyway** đơn giản hơn nếu chỉ dùng file SQL.

Yêu cầu trong project:

- **Migration (schema):** Tạo bảng `users` (và bảng role nếu tách riêng) qua changelog Liquibase (file XML/YAML/SQL trong `src/main/resources/db/changelog/`) hoặc qua Flyway (file SQL trong `src/main/resources/db/migration/`, đặt tên kiểu `V1__create_users_table.sql`). Khi dùng migration tool thì đặt `spring.jpa.hibernate.ddl-auto=validate` (hoặc `none`) để Hibernate không tự tạo/sửa schema; schema do migration quản lý.
- **Seed (dữ liệu ban đầu):** Tạo user ADMIN (username/password hash, role ADMIN) trong changelog Liquibase (changeSet `loadData` hoặc `sql`) hoặc file Flyway riêng (ví dụ `V2__seed_admin_user.sql`). Có thể kết hợp với `data.sql` của Spring Boot cho dev, nhưng dùng migration cho seed giúp prod và dev nhất quán.
- **Thứ tự:** Changelog master (Liquibase: `db.changelog-master.yaml` include từng file) hoặc quy ước đặt tên Flyway (V1, V2, …) để thứ tự chạy rõ ràng.

Tóm tắt: Dùng **Liquibase** (hoặc Flyway) để (1) tạo schema versioned, (2) seed user ADMIN và dữ liệu tham chiếu. Ở prod luôn dùng `ddl-auto=validate` và chạy migration khi deploy.

PHẦN 6 — CORS & API DOCUMENTATION (Swagger / OpenAPI)

6.1 CORS

- Cấu hình CORS để frontend (chạy domain/port khác) có thể gọi API: cho phép origin (ví dụ http://localhost:3000), methods GET/POST/PUT/PATCH/DELETE, allowed headers (Authorization, Content-Type), allowCredentials nếu cần. Cấu hình trong WebMvcConfigurer hoặc SecurityConfig.

6.2 Swagger UI & OpenAPI

- Thêm dependency SpringDoc OpenAPI (hoặc springfox) để có Swagger UI. Truy cập http://localhost:8080/swagger-ui.html (hoặc /swagger-ui/index.html tùy phiên bản).
- Cấu hình Bearer JWT trong OpenAPI: security scheme type HTTP Bearer, gắn vào các endpoint cần auth. Trên Swagger UI có thể nhập token và gọi API có bảo vệ.
- (Tùy chọn) Mô tả API, nhóm endpoint (Auth, Users, Admin).

CHECKLIST NỘP BÀI

- ☐ Đăng ký user mới, mật khẩu hash, role USER; đăng nhập trả JWT; gọi API với `Authorization: Bearer <token>` thành công.
- ☐ Không token hoặc token sai → 401; USER gọi API chỉ ADMIN → 403.
- ☐ GET /api/users có phân trang, sort, filter (search, role); GET /api/users/{id} trả 200 hoặc 404.

- ☐ POST /api/users (ADMIN) tạo user, validate, 201 và Location; PUT/PATCH chỉ chủ sở hữu hoặc ADMIN; DELETE chỉ ADMIN.
- ☐ Lỗi (validation, 404, trùng email) trả body thống nhất; không trả password trong bất kỳ response nào.
- ☐ Mỗi request có request ID trong MDC và header `X-Request-ID`; pattern log có requestId; MDC được clear sau request.
- ☐ AuthService và UserService có unit test; Repository có test @DataJpaTest; Controller có test ít nhất 200/201, 400 hoặc 404, 401, 403.
- ☐ application.yml có datasource, JPA, logging; có profile dev (H2) và prod (MySQL/validate, JWT từ env); JWT dùng @ConfigurationProperties.
- ☐ (Nếu làm phần migration) Schema và seed (user ADMIN) do Liquibase hoặc Flyway quản lý; ddl-auto=validate khi dùng migration.
- ☐ CORS cấu hình cho frontend; Swagger UI hoạt động và có Bearer JWT để test API có bảo vệ.

GỢI Ý TEST NHANH (cURL / PowerShell)

```
# 1. Đăng ký
curl -X POST http://localhost:8080/api/auth/register \
  -H "Content-Type: application/json" \
  -d "
{\"username\":\"user1\",\"email\":\"user1@test.com\",\"password\":\"Pass12
3!@\"}"

# 2. Đăng nhập (user1 hoặc admin)
curl -X POST http://localhost:8080/api/auth/login \
  -H "Content-Type: application/json" \
  -d "{\"usernameOrEmail\":\"admin\",\"password\":\"admin123\"}"

# 3. Gọi API có bảo vệ (thay TOKEN bằng accessToken từ bước 2)
curl -X GET "http://localhost:8080/api/users?page=0&size=5" \
  -H "Authorization: Bearer TOKEN"

# 4. Xóa user (chỉ ADMIN) → 204
curl -X DELETE http://localhost:8080/api/users/2 \
  -H "Authorization: Bearer TOKEN_ADMIN"
```

MỞ RỘNG (TÙY CHỌN)

- Refresh token (`POST /api/auth/refresh`), PATCH role (`PATCH /api/users/{id}/role`).
- Integration test: @SpringBootTest, luồng đăng ký → đăng nhập → gọi API với token.
- Structured logging (JSON) cho hệ thống log tập trung (Logstash/ELK).

Đề bài này độc lập với tài liệu lý thuyết; sinh viên chỉ cần làm đúng các yêu cầu và checklist trên để nộp bài.

Đánh giá toàn diện (checklist phủ kiến thức)

Dùng bảng dưới để tự kiểm tra đã phủ đủ chủ đề Spring Boot trong bài làm:

#	Chủ đề	Yêu cầu tối thiểu	Đã đạt
1	AuthN & AuthZ	Register, Login, JWT, role USER/ADMIN, 401/403, @PreAuthorize	☐
2	REST & CRUD	GET/POST/PUT/PATCH/DELETE users, phân trang, lọc, sort, DTO	☐
3	Entity, JPA, Repository	Entity User, JpaRepository, truy vấn theo điều kiện	☐
4	Exception & Validation	@RestControllerAdvice, ApiError thống nhất, Bean Validation	☐
5	Property & Config	application.yml, profiles dev/prod, @ConfigurationProperties (JWT)	☐
6	Logging	Request ID (MDC), X-Request-ID header, pattern log, clear MDC	☐
7	Testing	Unit test Service (Mockito), @DataJpaTest, MockMvc (401/403)	☐
8	CORS & Swagger	CORS, Swagger UI, Bearer JWT trong OpenAPI	☐
9	DB Migration & Seed	Liquibase hoặc Flyway: schema + seed (admin), ddl-auto=validate	☐ (tùy chọn)

Nộp bài: Theo hướng dẫn giảng viên (thường gồm: mã nguồn, README hướng dẫn chạy, có thể kèm báo cáo ngắn hoặc checklist trên đã tick).